

CALENDRIA – Méthodologie et Organisation du Projet

Projet : CALENDRIA–AssistantdeRéservationIntelligentMulti-Canal

Auteur : BOUGHERARA Safi

Formation : CDA – Concepteur Développeur d'Applications

Date : Février 2026

Jalon : 2 – Méthodologie de Projet & Conception UI/UX

Table des Matières

- 1. [Méthode de Gestion de Projet](#)
- 2. [Planning Global \(Macro-Planning\)](#)
- 3. [Outils de Suivi](#)
- 4. [Gestion du Code Source \(Git\)](#)
- 5. [CI/CD Planifié](#)

1. Méthode de Gestion de Projet

1.1 Méthodologie Retenue : Scrum Adapté Solo

Choix : Méthode Agile Scrum, adaptée pour un développeur unique.

Justification du Choix

Critère	Cycle en V	Kanban	Scrum Adapté	Choix
Flexibilité	Rigide, phases séquentielles	Très flexible, pas de cadre temporel	Flexible avec cadre temporel (sprints)	✅ Scrum
Correspondance jalons	Bon pour planification linéaire	Pas de jalons naturels	Sprints = Jalons mensuels	✅ Scrum
Gestion des risques	Détection tardive	Détection correcte	Détection précoce (revues de sprint)	✅ Scrum
Adaptation solo	Bien adapté	Bien adapté	Nécessite adaptation	⚠️ Acceptable
Suivi d'avancement	Jalons séquentiels	Flux continu (WIP limits)	Incréments livrables à chaque sprint	✅ Scrum

Conclusion : Scrum est la méthodologie la plus adaptée car :

- 1. **Correspondance naturelle avec les jalons :** Chaque sprint mensuel correspond exactement à un jalon du projet (Janvier → Juin), ce qui donne un cadre temporel clair et mesurable.
- 2. **Livraison incrémentale :** Chaque sprint produit un incrément fonctionnel démontrable, permettant de valider progressivement l'avancement avec le référent.
- 3. **Capacité d'adaptation :** Les revues de sprint permettent d'ajuster le backlog en fonction des retours et des imprévus techniques (ex : intégration Twilio, quota OpenAI).
- 4. **Transparence :** Le backlog, le burndown chart et les revues offrent une visibilité claire sur l'avancement du projet.

Adaptation Scrum pour un Développeur Solo

Le framework Scrum est conçu pour des équipes de 3 à 9 personnes. En tant que développeur unique, j'ai adapté les cérémonies et rôles comme suit :

Élément Scrum	Version Équipe	Adaptation Solo
Product Owner	Client/Partie prenante	Moi-même (backlog géré via GitHub Issues)
Scrum Master	Facilitateur dédié	Moi-même (auto-discipline, respect du cadre)
Dev Team	3-9 développeurs	Moi-même
Sprint Planning	Réunion d'équipe (2-4h)	Planification en début de mois (~1h), backlog sprint défini dans GitHub Projects
Daily Standup	Réunion quotidienne (15 min)	Journal de bord quotidien : 3 questions (Qu'ai-je fait ? Que vais-je faire ? Obstacles ?) notées dans un fichier JOURNAL.md
Sprint Review	Démonstration au PO	Présentation au référent lors des jalons + auto-revue des livrables
Sprint Retrospective	Réunion d'amélioration (1h)	Auto-analyse en fin de sprint : Ce qui a bien fonctionné / Ce qui peut être amélioré / Actions correctives
Durée du Sprint	1-4 semaines	1 mois (aligné sur les jalons)
Definition of Done	Critères d'équipe	Critères personnels : code testé, documenté, mergé dans develop, livrable jalon prêt

1.2 Organisation des Sprints

Le projet est découpé en **6 sprints mensuels**, chacun correspondant à un jalon :

Sprint	Période	Jalon	Objectif Principal	Livrable
Sprint 1	Janvier 2026	Jalon 1	Cadrage du projet	CDCF complet 
Sprint 2	Février 2026	Jalon 2	Méthodologie + Design	Document méthodo + Maquettes UI/UX
Sprint 3	Mars 2026	Jalon 3	Conception technique	MCD/MLD/MPD + API REST + Algorithme tables
Sprint 4	Avril 2026	Jalon 4	Développement cœur	Chatbot WhatsApp + Dashboard restaurateur + UML
Sprint 5	Mai 2026	Jalon 5	Développement complémentaire	Widget Web + SMS + Tests + Sécurité
Sprint 6	Juin 2026	Jalon 6	Finalisation	Déploiement + Documentation finale

1.3 Backlog Produit

Le backlog produit est organisé en **User Stories** priorisées selon la méthode **MoSCoW** :

Must Have (Obligatoire)

- En tant que **client**, je veux réserver une table via WhatsApp pour ne pas avoir à appeler le restaurant.
- En tant que **restaurateur**, je veux voir toutes les réservations du jour sur un dashboard pour organiser mon service.
- En tant que **restaurateur**, je veux configurer mes tables et horaires pour que le système connaisse ma capacité.
- En tant que **client**, je veux recevoir une confirmation SMS immédiate pour être sûr que ma réservation est prise en compte.
- En tant que **restaurateur**, je veux pouvoir annuler ou modifier une réservation depuis le dashboard.

Should Have (Important)

- En tant que **client**, je veux réserver via un widget web sur le site du restaurant pour avoir une alternative à WhatsApp.
- En tant que **restaurateur**, je veux consulter les statistiques de remplissage pour optimiser mon activité.
- En tant que **client**, je veux recevoir un rappel SMS 24h avant pour ne pas oublier ma réservation.
- En tant que **restaurateur**, je veux une fiche client avec historique pour personnaliser l'accueil.

Could Have (Souhaitable)

- En tant que **client**, je veux réserver par SMS direct pour ne pas avoir besoin de WhatsApp.
- En tant que **restaurateur**, je veux un plan de salle visuel pour gérer les tables graphiquement.
- En tant que **restaurateur**, je veux exporter les données en CSV/PDF pour les analyser.

Won't Have (Hors scope)

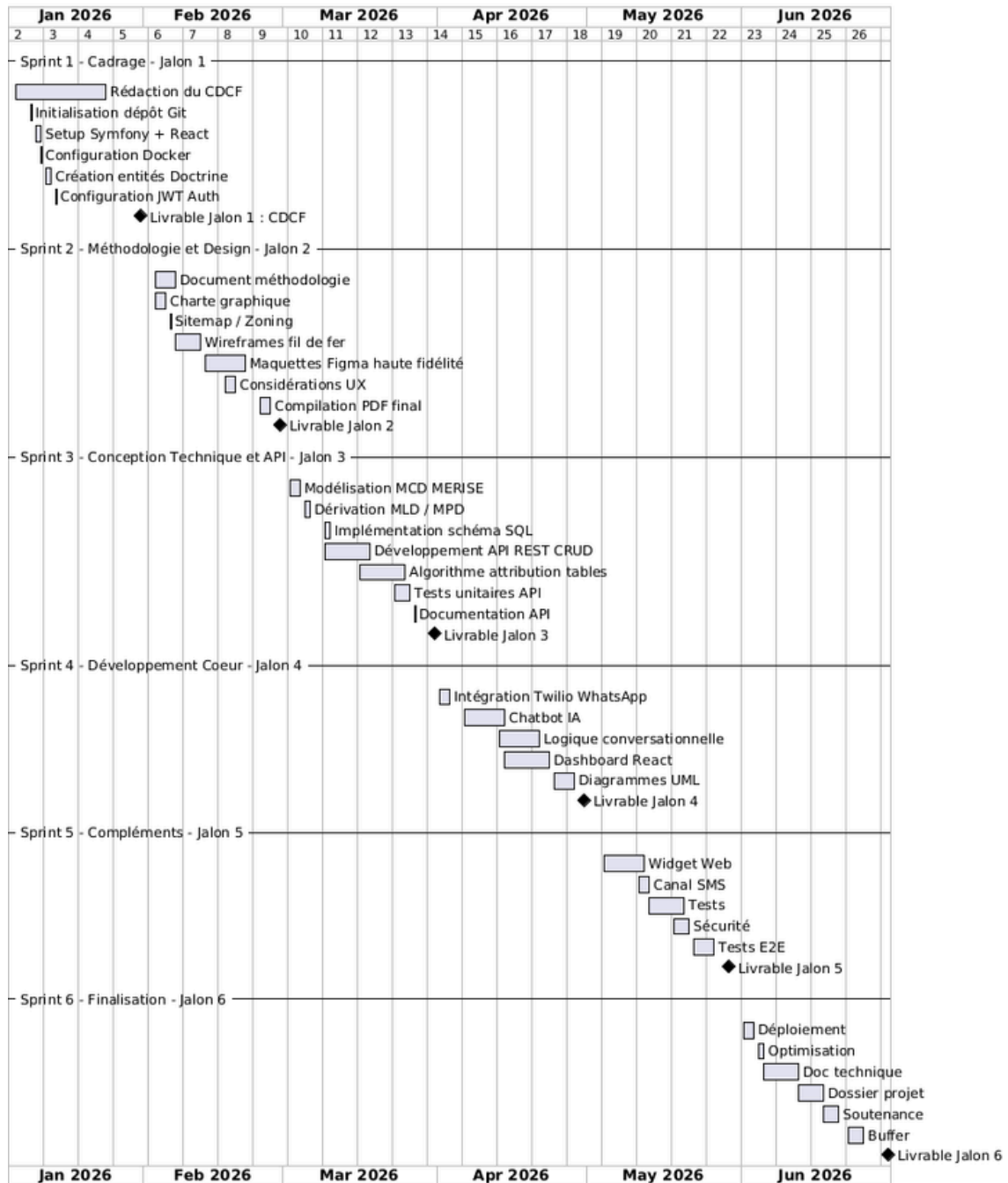
- Paiement en ligne, commande de plats, multi-langues, application mobile native.

2. Planning Global (Macro-Planning)

2.1 Diagramme de Gantt

Le diagramme de Gantt ci-dessous présente le planning global du projet de Janvier à Juin 2026.

CALENDRIA - Planning Global du Projet (Janvier - Juin 2026)



2.2 Détail des Phases

Phase 1 – Cadrage (Janvier 2026) ✓ TERMINÉ

Tâche	Durée	Statut
Rédaction du CDCF	3 semaines	✓ Terminé
Initialisation du dépôt Git	1 jour	✓ Terminé
Setup projet Symfony + React	2 jours	✓ Terminé
Configuration Docker	1 jour	✓ Terminé
Création des entités Doctrine	2 jours	✓ Terminé
Configuration JWT Auth	1 jour	✓ Terminé

Livrables : CDCF validé, dépôt Git initialisé, structure projet en place.

Phase 2 – Méthodologie & Design (Février 2026) ← SPRINT ACTUEL

Tâche	Durée
Document méthodologie	1 semaine
Charte graphique	2 jours
Sitemap / Zoning	1 jour
Wireframes	3 jours
Maquettes haute fidélité (Figma)	1 semaine
Considérations UX	2 jours

Livrables : Document méthodologie, maquettes UI/UX.

Phase 3 – Conception Technique & API (Mars 2026)

Tâche	Durée
Modélisation MCD (MERISE)	3 jours
Dérivation MLD/MPD	2 jours
Implémentation schéma SQL + migrations	2 jours
Développement API REST (CRUD complet)	1 semaine
Algorithme d'attribution des tables	1 semaine
Tests unitaires API (PHPUnit)	3 jours
Documentation API (Swagger/OpenAPI)	1 jour

Livrables : MCD/MLD/MPD, API REST fonctionnelle, algorithme validé.

Phase 4 – Développement Cœur (Avril 2026)

Tâche	Durée
Intégration Twilio WhatsApp Sandbox	3 jours
Chatbot IA (OpenAI GPT-4o-mini)	1 semaine
Logique conversationnelle (collecte infos)	1 semaine
Dashboard restaurateur (React)	1 semaine
Diagrammes UML (classes, séquence, use case)	3 jours

Livrables : Chatbot WhatsApp fonctionnel, Dashboard V1, Diagrammes UML.

Phase 5 – Compléments & Qualité (Mai 2026)

Tâche	Durée
Widget Web chatbot (React component)	1 semaine
Canal SMS direct (Twilio SMS)	3 jours
Tests unitaires + fonctionnels	1 semaine
Sécurité OWASP + RGPD	3 jours
Tests E2E (Cypress optionnel)	2 jours

Livrables : Widget Web, SMS, couverture de tests > 80%.

Phase 6 – Finalisation (Juin 2026)

Tâche	Durée
Déploiement Docker production	3 jours
Optimisation performances	2 jours
Documentation technique finale	1 semaine
Dossier projet complet	3 jours
Préparation soutenance	3 jours
Buffer pour imprévus	3 jours

Livrables : Application déployée, documentation complète, dossier final.

3. Outils de Suivi

3.1 GitHub Projects (Kanban Board)

L'avancement du projet est suivi via **GitHub Projects**, configuré en mode Kanban avec les colonnes suivantes :

Colonne	Description	Exemple
 Backlog	Toutes les tâches identifiées mais non planifiées	"Implémenter export CSV"
 Sprint Backlog	Tâches planifiées pour le sprint en cours	"Créer wireframes dashboard"
 En cours	Tâche activement développée	"Rédiger document méthodologie"
 En revue	Tâche terminée en attente de relecture	"Revoir charte graphique"
 Terminé	Tâche validée et mergée	"CDCF Jalon 1"

3.2 GitHub Issues

Chaque tâche du backlog est créée sous forme de **GitHub Issue** avec :

- **Titre** descriptif (ex : [JALON2]Rédigerdocument méthodologie)
- **Labels** : jalon-2, documentation , priorité-haute

- **Milestone** : Lié au jalon correspondant (ex : Jalon 2 - Février)
- **Description** : Critères d'acceptation et sous-tâches (checklist)

3.3 Fréquence de Suivi

Activité	Fréquence	Support
Journal de bord (Daily)	Quotidien	JOURNAL.md dans le dépôt
Mise à jour Kanban	À chaque changement de statut	GitHub Projects
Revue de sprint	Fin de mois (= fin de jalon)	Présentation au référent
Rétrospective	Fin de mois	Section dédiée dans JOURNAL.md
Adaptation du planning	Si retard détecté (> 2 jours)	Mise à jour Gantt + Issues

3.4 Indicateurs de Suivi

- **Vélocité** : Nombre de story points complétés par sprint
- **Burndown Chart** : Suivi visuel de l'avancement au sein d'un sprint (via GitHub Projects)
- **Taux de complétion** : % de tâches terminées par rapport au sprint backlog
- **Retard accumulé** : Nombre de jours de retard par rapport au planning initial

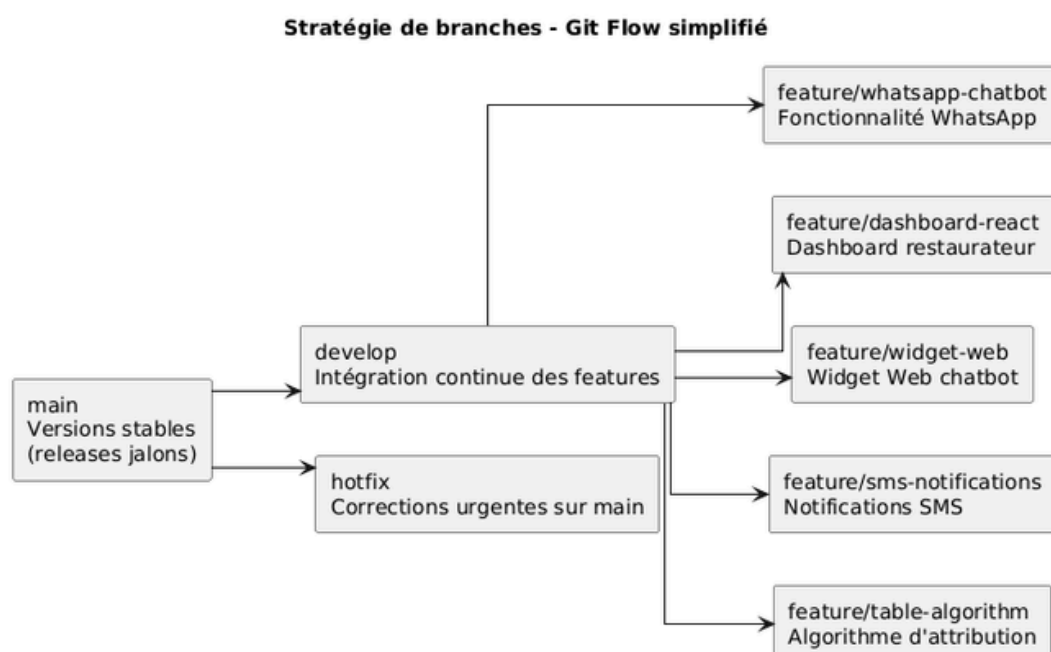
4. Gestion du Code Source (Git)

4.1 Dépôt Git

- **Plateforme** : GitHub
- **Dépôt** : https://github.com/SafiBougherara/projet_fin_annee
- **Visibilité** : Public
- **Statut actuel** : 20+ commits, structure projet en place (Symfony + React + Docker)

4.2 Stratégie de Branches

Le projet suit le modèle **Git Flow simplifié** :



Règles de Branches

Branch	Source	Destination	Usage
main	-	-	Code de production, merge uniquement depuis develop à chaque jalon
develop	main	main	Intégration continue, branche par défaut pour le développement
feature/*	develop	develop	Développement d'une fonctionnalité isolée
hotfix/*	main	main + develop	Correction urgente d'un bug en production

4.3 Convention de Commits

Les messages de commit suivent la convention **Conventional Commits** :

```
<type>(<scope>): <description>
```

Types :

- feat: Nouvelle fonctionnalité
- fix: Correction de bug
- docs: Documentation
- style: Formatage (pas de changement de logique)
- refactor: Refactoring du code
- test: Ajout/modification de tests
- chore: Tâches de maintenance (CI, config, etc.)

Exemples :

- feat(api): Add reservation CRUD endpoints
- fix(chatbot): Fix date parsing for French format
- docs(jalon2): Add methodology document
- test(api): Add unit tests for table algorithm

4.4 Politique de Merge

Même en développement solo, les bonnes pratiques suivantes sont appliquées :

1. **Pas de commit direct sur main** : Tout passe par develop via merge
2. **Feature branches** : Chaque fonctionnalité est développée dans une branche dédiée
3. **Auto-revue de code** : Relecture du diff avant chaque merge (git diff develop..feature/xxx)
4. **Squash merge** : Regroupement des commits d'une feature en un seul commit propre lors du merge dans develop
5. **Tags de version** : Tag Git à chaque jalon (ex : v0.2.0 pour Jalon 2)

4.5 Historique Actuel

Le dépôt contient déjà les éléments suivants (travail du Sprint 1) :

- ☒ Structure du projet (backend/frontend/docker/documentation)
 - ☒ Backend Symfony 6.4 avec 5 entités Doctrine (Restaurant, Table, Client, Reservation, Service)
 - ☒ API Platform configurée avec endpoints REST
 - ☒ Authentification JWT (LexikJWTAuthenticationBundle)
 - ☒ Frontend React 18 + TypeScript + Vite
 - ☒ Docker Compose (PostgreSQL, Backend, Frontend, Adminer)
 - ☒ CDCF complet (Jalon 1)
-

5. CI/CD Planifié

5.1 Vue d'Ensemble de la Pipeline

La pipeline CI/CD sera mise en place via **GitHub Actions** pour automatiser les tests, la qualité du code et le déploiement.



5.2 Détail de la Pipeline CI

Fichier : `.github/workflows/ci.yml`

Déclencheurs :

- Chaque `push` sur `main` et `develop`
- Chaque `pull_request` vers `main` et `develop`

Jobs prévus :

Job 1 : Analyse Statique (Lint)

Outil	Cible	Fonction
PHPStan (niveau 6)	Backend PHP	Analyse statique, détection de bugs potentiels
ESLint	Frontend TypeScript	Qualité du code, conventions
Prettier	Frontend	Formatage consistant

Job 2 : Tests Automatisés

Type	Outil	Couverture Cible
Tests unitaires backend	PHPUnit	> 80% des classes métier
Tests fonctionnels API	PHPUnit (WebTestCase)	Tous les endpoints CRUD
Tests unitaires frontend	Jest + React Testing Library	> 70% des composants

Job 3 : Build Docker

Action	Description
--------	-------------

Build image backend	PHP 8.2 + Symfony + dépendances Composer
Build image frontend	Node 20 + React + build de production
Test docker-compose	Vérification que l'ensemble des services démarre correctement

5.3 Pipeline CD (Déploiement)

Le déploiement automatisé sera mis en place **à partir du Sprint 5 (Mai)** :

Étape	Action	Déclencheur
Build images	Construction des images Docker optimisées (multi-stage)	Tag de release (v*)
Push Docker Hub	Publication des images sur Docker Hub	Après build réussi
Déploiement staging	Déploiement automatique sur environnement de test	Push sur develop
Déploiement production	Déploiement sur serveur de production	Tag de release (v*) sur main

Hébergement envisagé :

- **Option 1** : Heroku (gratuit avec limite) – Simple, idéal pour le MVP
- **Option 2** : Railway.app – Supporté Docker, gratuit pour le développement
- **Option 3** : VPS (OVH/Hetzner) – Plus de contrôle, coût ~5€/mois

5.4 Outils DevOps Prévus

Outil	Usage	Statut
GitHub Actions	Pipeline CI/CD	🕒 À configurer (Sprint 3)
Docker	Conteneurisation	✅ En place
Docker Compose	Orchestration locale	✅ En place
Docker Hub	Registre d'images	🕒 À configurer (Sprint 5)
PHPUnit	Tests backend	🕒 À écrire (Sprint 3)
Jest	Tests frontend	🕒 À écrire (Sprint 5)
PHPStan	Analyse statique PHP	🕒 À configurer (Sprint 3)
ESLint	Linting TypeScript	✅ En place

5.5 Calendrier d'Implémentation DevOps

Sprint	Actions DevOps
Sprint 2 (Fév)	Documentation de la stratégie CI/CD (ce document)
Sprint 3 (Mars)	Configuration GitHub Actions (lint + tests backend)

Sprint 4 (Avr)	Ajout tests fonctionnels API, build Docker dans la CI
Sprint 5 (Mai)	Tests frontend, pipeline CD, déploiement staging
Sprint 6 (Juin)	Déploiement production, monitoring

Annexes

A. Liens Utiles

Dépôt GitHub : https://github.com/SafiBougherara/projet_fin_annee

B. Glossaire

Terme	Définition
Sprint	Itération de développement à durée fixe (1 mois dans notre cas)
Backlog	Liste ordonnée de toutes les fonctionnalités souhaitées
User Story	Description d'une fonctionnalité du point de vue de l'utilisateur
Definition of Done	Critères à remplir pour considérer une tâche comme terminée
CI/CD	Intégration Continue / Déploiement Continu
MoSCoW	Méthode de priorisation (Must/Should/Could/Won't)
Git Flow	Stratégie de gestion des branches Git